

Scheduling jobs on parallel machines with setup times and ready times

Michele Pfund ^{b,1}, John W. Fowler ^{a,*}, Amit Gadkari ^a, Yan Chen ^a

^a Department of Industrial Engineering, Arizona State University, P.O. Box 5906, Tempe, AZ 85287-5906, USA

^b Department of Supply Chain Management, Arizona State University, P.O. Box 4706, Tempe, AZ 85287-4706, USA

Received 17 December 2003; received in revised form 31 October 2006; accepted 17 August 2007

Available online 4 November 2007

Abstract

In this research we are interested in scheduling jobs with ready times on identical parallel machines with sequence dependent setups. Our objective is to minimize the total weighted tardiness. As this problem is NP-Hard, we develop a heuristic to solve this problem in reasonable time. Our approach is an extension of the apparent tardiness cost with setups (ATCS) approach by [Lee, Y. H., Pinedo, M. (1997). Scheduling jobs on parallel machines with sequence dependent setup times. *European Journal of Operational Research*, 100, 464–474.] to allow non-ready jobs to be scheduled – meaning we allow a machine to remain idle for a high priority job arriving at a later time. To determine the scaling parameters for our composite dispatching rule (called ATCSR), we first develop a ‘grid approach’ that considers multiple values for the scaling parameters, generates multiple schedules, and chooses the best schedule for the solution. This experimentation was then used to develop regression equations to predict the values of the scaling parameters that would yield the highest quality solution. The grid and regression versions of ATCSR provide better performance than grid and empirically based formula versions of ATCS, BATCS, and X-RM which are the prominent algorithms in the literature.

© 2007 Elsevier Ltd. All rights reserved.

Keywords: Scheduling; Heuristics; Parallel machines; Setup time; Ready time

1. Introduction

The semiconductor industry is highly competitive and is marked by high initial capital investment and short product lifecycles. This puts a demand of low time to market and high on-time delivery of products with limited capacity of equipment. This is further complicated by frequent equipment failures and reentrant flow of products. To achieve the goal of on-time delivery, research is being conducted on scheduling wafer fabrication facilities using algorithms such as the Shifting Bottleneck Heuristic (Mason, Fowler, & Carlyle, 2002). The

* Corresponding author. Tel.: +1 480 965 3727; fax: +1 480 965 8692.

E-mail addresses: michele.pfund@asu.edu (M. Pfund), john.fowler@asu.edu (J.W. Fowler), gadman10@hotmail.com (A. Gadkari), jenny.yan.chen@gmail.com (Y. Chen).

¹ Tel.: +1 480 965 6409.

quality of the schedule generated by this heuristic is dependent on the quality of the schedule of individual toolgroups. We develop a heuristic for scheduling a toolgroup of identical parallel machines with an objective of minimizing the total weighted tardiness. We specifically pay attention to the phenomenon that due to unequal weights and setup times of the jobs, there could be a better schedule in which non-ready jobs are scheduled even when there are ready jobs available.

2. Problem definition

We consider a toolgroup of m identical machines in parallel denoted by Pm . Each of the machines can process only one job at a time. There are a total of n jobs arriving at different times to the toolgroup. Each job j has a processing time p_j , arrival time or ready time r_j , due date d_j , and a weight, w_j . When a job k is processed after job j a setup time s_{jk} is incurred. This setup time is solely dependent on jobs j and k and is independent of the machine on which the jobs are being processed. The completion time of job j is denoted by C_j , and the tardiness of job j is defined as $T_j = \max(C_j - d_j, 0)$. Using the $\alpha|\beta| \gamma$ notation of Lawler et al. (1982), the representation of the problem being researched is $Pm|r_j, s_{jk}|\Sigma w_j T_j$.

3. Previous related work

Scheduling problems with total tardiness as the objective function are NP-Hard even for a single machine scheduling environment (Du & Leung, 1990). As the single machine problem reduces to the parallel machine total weighted tardiness problem, the parallel machine problem is NP-Hard.

Abdul-Razaq, Potts, and Van Wassenhove (1990) presented a survey of the algorithms that use branch and bound or dynamic programming based approaches to get an exact solution as well as approaches for finding a lower bound for the single machine total weighted tardiness problem ($I|\Sigma w_j T_j$). They survey dynamic programming approaches by Lawler (1979), Schrage and Baker (1978), lower bounding approaches by Rinnooy Kan, Lageweg, and Lenstra (1975), Potts and Van Wassenhove (1985), Fisher (1981) and others. Other attempts to solve the total weighted tardiness problem include one by Huegler and Vasko (1997) who developed an approach which generates an initial schedule using a priority index, followed by piecewise dynamic programming for the single machine total weighted tardiness problem ($I|\Sigma w_j T_j$). Luh, Hoitomt, Max, and Pattipati (1990) proposed a Lagrangian dual relaxation approach to generate a listing of jobs followed by a greedy approach to form a feasible schedule for the parallel machine problem ($Pm|s_{jk}|\Sigma w_j T_j$). Eom, Shin, Kwun, Shim, and Kim (2002) proposed a 3-phase heuristic, made up of pre-processing by earliest due date, grouping and sequencing jobs according to setup types improved by tabu search and allocating jobs to machines, for the problem ($Pm|s_{jk}|\Sigma w_j T_j$).

While enumerative approaches provide exact solutions, they are highly computationally intensive for large problem instances. For example, for a 30-job $I|\Sigma w_j T_j$ scheduling problem, Abdul-Razaq et al. (1990) discussed a branch and bound algorithm that consumes more than 60 s. Thus, for large problem instances they may not be useful for scheduling in real world manufacturing.

The more common approach in industry is to use dispatching rules. Dispatching rules are the easiest way to address the parallel machine weighted tardiness problem and have been described by Pinedo (2002). Some of the dispatching rules in use are earliest due date (EDD), shortest processing time (SPT), weighted shortest processing time (WSPT), least slack (LS) and critical ratio (CR). Although these rules are very fast, they are myopic and the quality of the solution is typically much worse than the optimal.

To overcome this problem composite rules have been developed by combining multiple dispatching rules. Two of the early composite dispatching rules were the COVERT rule by Carroll (1965) and the modified operation due date rule (MOD) by Baker and Bertrand (1982). Vepsalainen and Morton (1987) applied the principles of the COVERT and the MOD rule to develop the apparent tardiness cost heuristic (ATC) for the parallel machine total weighted tardiness problem ($Pm|\Sigma w_j T_j$). Alidaee and Rosa (1997) proposed the modified due date heuristic (MDD) for the parallel machine problem ($Pm|\Sigma w_j T_j$). For scheduling parallel machines with setups ($Pm|s_{jk}|\Sigma w_j T_j$), Lee and Pinedo (1997) built upon the ATC approach and developed a 3-phase approach consisting of identifying problem instance characteristics, finding an initial schedule using the apparent tardiness cost with setups rule followed by simulated annealing to improve the solution. Mason

et al. (2002) modify the ATCS approach for use in batch machines ($Jm|r_j, s_{jk}, batch, recrc|\Sigma w_j T_j$) to allow non-ready jobs to be scheduled; i.e., the machine may remain idle for a job becoming ready in the near future even if other jobs are available for processing at the time when the machine becomes free.

Rachamadugu and Morton (1982) proposed the RM heuristic for parallel machine scheduling. Morton and Pentico (1993) modified the RM heuristic to allow consideration of jobs arriving at a later time and develop their X-RM heuristic. Morton and Ramnath (1995) present extensive comparisons with other heuristics while presenting their X-dispatch heuristic, which allows non-ready jobs to be scheduled.

There have been attempts to determine the parameters in the ATC index. Lee and Pinedo (1997) use a heuristic approach to estimate the parameters used in the ATCS index. Recently Park, Kim, and Lee (2000) used neural networks to estimate the values of scaling parameters and obtained some improvement over the heuristic approach.

Several variations of the ATC rule are summarized in Table 1 and described in the remainder of this section.

3.1. Apparent tardiness cost with setups (ATCS)

For the apparent tardiness cost with setups (ATCS) rule, the index given by Lee and Pinedo (1997) is used. Whenever a machine becomes free, calculate the ATCS index for each of the non-processed jobs ready at that time. The index is determined by multiplying the corresponding terms in Table 1. The job with the highest ATCS index is chosen to be scheduled next on the machine. If none of the jobs is ready when the next machine becomes free, the machine free time t is advanced to the earliest ready time amongst the non-processed jobs and then the index is calculated for all the jobs ready at this time.

3.2. Batch apparent tardiness cost with setups (BATCS)

For the batch apparent tardiness cost with setups and ready times (BATCS) rule, the index given by Mason et al. (2002) with a batch size of 1 is used. This index allows ready times to be considered directly by allowing non-ready jobs to be scheduled. Whenever a machine becomes free, calculate the BATCS index, given by multiplying the corresponding terms in Table 1, for each of the non-processed jobs including those which are not ready at that time. Schedule the job with the highest BATCS index as the next job to be processed.

3.3. Batch apparent tardiness cost with setups and modified (BATCSmod)

The BATCS rule was modified as follows. The way in which the slack term in the BATCS index handles ready times was changed to determine if there is any significant impact on the results. The BATCSmod index is given by

Table 1
Comparison of terms in ATC based approaches

Rule	WSPT term	Slack term	Setup term	Ready term
ATC	$\frac{w_j}{p_j}$	$\exp\left(-\frac{\max(d_j - p_j - t, 0)}{k_1 p}\right)$	NA	NA
ATCS	$\frac{w_j}{p_j}$	$\exp\left(-\frac{\max(d_j - p_j - t, 0)}{k_1 p}\right)$	$\exp\left(-\frac{s_{ij}}{k_2 s}\right)$	NA
BATCS	$\frac{w_j}{p_j}$	$\exp\left(-\frac{\max(d_j - p_j + r_j - t, 0)}{k_1 p}\right)$	$\exp\left(-\frac{s_{ij}}{k_2 s}\right)$	NA
BATCSmod	$\frac{w_j}{p_j}$	$\exp\left(-\frac{\max(d_j - p_j + \max(r_j - t, 0), 0)}{k_1 p}\right)$	$\exp\left(-\frac{s_{ij}}{k_2 s}\right)$	NA
X-RM	$\frac{w_j}{p_j}$	$\exp\left(-\frac{\max(d_j - p_j - t, 0)}{k_1 p}\right)$	NA	$\left(1 - \frac{B}{P_{\min}}(\max(r_j - t, 0))\right)$
X-RMmod	$\frac{w_j}{p_j}$	$\exp\left(-\frac{\max(d_j - p_j - t, 0)}{k_1 p}\right)$	$\exp\left(-\frac{s_{ij}}{k_2 s}\right)$	$\left(1 - \frac{B}{P_{\min}}(\max(r_j - t, 0))\right)$
ATCSR	$\frac{w_j}{p_j}$	$\exp\left(-\frac{\max(d_j - p_j - \max(r_j, t), 0)}{k_1 p}\right)$	$\exp\left(-\frac{s_{ij}}{k_2 s}\right)$	$\exp\left(-\frac{\max(r_j - t, 0)}{k_3 p}\right)$

$$I_{\text{BATCS mod}}(t, l) = \frac{w_j}{p_j} \exp\left(\frac{-\max(d_j - p_j + \max(r_j - t, 0), 0)}{k_1 \bar{p}}\right) \exp\left(-\frac{s_{lj}}{k_2 \bar{s}}\right)$$

Schedule the job with the highest BATCSmod index as the next job to be processed on that machine.

3.4. X-dispatch Rachamadugu and Morton (X-RM)

The X-dispatch Rachamadugu and Morton (X-RM) by Morton and Pentico (1993) attempts to determine if there is benefit in waiting for non-ready jobs. The index can be multiplied from the terms in Table 1, where $p_{\min}$ is the minimum of processing times amongst the ready jobs and B is a constant to vary the amount of importance given to late jobs. Whenever a machine becomes free, calculate the X-RM index for each of the non-processed jobs including those which are not ready at that time, but would be ready before $t + p_{\min}$ where t is the time when the machine becomes free. Schedule the job with the highest X-RM index as the next job on the machine.

3.5. X-dispatch Rachamadugu and Morton modified (X-RMmod)

We modified the X-RM index to account for setup times by inserting a setup term in the product, identical with the setup term in the ATCS index. The rest of the procedure is the same as X-RM index.

4. Methodology

Our approach is based on ATCS by Lee and Pinedo (1997). Although ready times are implicitly dealt with in the ATCS approach by only considering jobs that are ready when deciding what to process next, it does not explicitly deal with ready times in the index. The BATCS rule accounts for the ready times in the slack term. The X-RM rule discounts ready times with a linear term. Our approach explicitly considers a separate exponential term for the ready times which decays faster than a linear term and also includes ready times in the slack term of the index which makes the slack term better related to the job slack. The remainder of this section describes the ATCSR index and the scaling parameter associated with it.

4.1. Apparent tardiness cost with setups and ready times index (ATCSR index)

The ATCSR index is given by

$$I_{\text{ATCSR}}(t, l) = \frac{w_j}{p_j} \exp\left(\frac{-\max(d_j - p_j - \max(r_j, t), 0)}{k_1 \bar{p}}\right) \exp\left(-\frac{s_{lj}}{k_2 \bar{s}}\right) \exp\left(-\frac{\max(r_j - t, 0)}{k_3 \bar{p}}\right)$$

The ATCSR index is a priority index to estimate the urgency of scheduling that job. The job with the highest ATCSR index is considered to have the highest priority. It is a composite of the four terms shown in the last row of Table 1. The weighted shortest processing time (WSPT) term gives preference to jobs with a high weight (importance) and a short processing time.

In the slack term, $d_j - p_j - \max(r_j, t)$ is the slack remaining for the processing of the job. If the job is already late, the exponent goes to 0, thus making the term equal to 1. When slack is remaining for a job, the slack term assumes a value lower than 1 depending on the amount of slack remaining and the value of the scaling parameter k_1 . A lower value of the scaling parameter k_1 amplifies the effect of this term.

The setup term becomes smaller as the setup time becomes longer, thus discouraging scheduling of jobs with higher setup when scheduled after the present job. A lower value of the scaling parameter k_2 amplifies the effect of this term – meaning that it discourages setups to a greater extent.

Finally, the ready time term is unity when the job is ready at current time t and is less than one when the job is not ready. A lower value of k_3 amplifies the effect of this term, meaning it discourages the scheduling of jobs

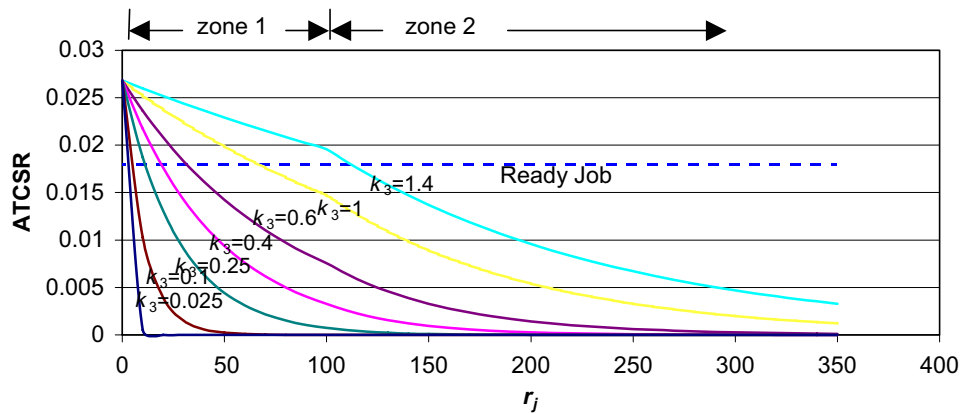


Fig. 1. Variation of ATCSR versus r_j for different values of k_3 .

that are not ready within a short interval from the current time. In the limit, when k_3 becomes very small only those jobs that are ready are scheduled.

Fig. 1 was constructed to demonstrate the impact of k_3 . In this simple example, the machine is available at time 0 and the slack for the job is initially positive and then becomes negative. The X axis considers the effect as the ready time of the job increases. From the above figure we can see that the region for r_j can be divided into two zones:

Zone 1: Slack is positive. In this region the slack term increases and the ready term decreases as the ready time increases. The change of the ATCSR index in this zone depends on both the k_1 and k_3 values.

Zone 2: Slack is negative. In this region, the slack term reaches one and the ready term decreases as the ready time increases. The decreasing speed of ATCSR index only depends on the k_3 value.

In Fig. 1, if the ATCSR of the non-ready job is at a point higher than that of the ready job, indicated by the dotted line, the decision is to wait for the job which is yet to arrive. Whereas if it is below the dotted line, then the ready job is chosen. Thus, the choice of k_3 has an impact on how long we are going to wait for a non-ready job, i.e., a high value of k_3 means one will wait longer for a non-ready job.

4.2. Effect of scaling parameters

From the section above, it is obvious that the ATCSR index can be managed to give different importance to each of the four terms by varying the values of the scaling parameters. In fact, scaling parameters k_1 , k_2 and k_3 have a large impact on the quality of solution and this is expected as they vary the importance given to the terms in the index.

Although the normalizing terms for the scaling parameters k_1 and k_2 are obvious (average processing time and average setup time, respectively), the normalizing term for the scaling parameter k_3 deserves a mention. Average processing time was chosen over makespan or average ready time as it is a better indicator of how much inserted idle time should be allowed for the machine in a particular schedule. Other possible terms like makespan increase as the number of jobs in the schedule increases whereas mean processing time remains the same even if the number of jobs increases.

5. ATCSR algorithm using grid approach

As mentioned previously, the total weighted tardiness of the schedule generated by using the ATCSR index is greatly influenced by the values of the scaling parameters. One of the solutions to this problem is to generate a schedule using a predetermined value of scaling parameters and then use simulated annealing or genetic algorithms to improve the quality of the solution, which is proposed for ATCS by Lee and Pinedo (1997).

We propose a grid search method that generates multiple schedules. Then the schedule with the lowest total weighted tardiness can be selected from these schedules. Thus, this method does a search of the solution space and usually gives a schedule with total weighted tardiness lower than that given by generating the schedule with a single set of values for scaling parameters.

5.1. Combinations of scaling parameters for grid approach

The grid used for comparable algorithms such as ATCS (Lee & Pinedo, 1997) was studied and additional values were added to ensure that the frequently occurring values were not the extreme values in the grid. We choose 22 values for k_1 , 11 values for k_2 , and 13 values for k_3 . We form a combination of scaling parameters by selecting one value each of k_1 , k_2 and k_3 from the list. Thus, there are a total of 3146 combinations.

Values for k_1 : {0.2, 0.6, 0.8, 1, 1.2, 1.4, 1.6, 1.8, 2, 2.4, 2.8, 3.2, 3.6, 4.0, 4.4, 4.8, 5.2, 5.6, 6.0, 6.4, 6.8, 7.2}

Values for k_2 : {0.1, 0.3, 0.5, 0.7, 0.9, 1.1, 1.3, 1.5, 1.7, 1.9, 2.1}

Values for k_3 : {0.001, 0.0025, 0.004, 0.005, 0.025, 0.04, 0.05, 0.25, 0.4, 0.6, 0.8, 1, 1.2}

Since we will compare the performance of ATCSR to X-RMmod, we note that the grid for X-RMmod will use the same values as above for k_1 , and k_2 and the following values for B :

{1.35, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2.0, 2.1, 2.15, 2.2, 2.25, 2.3}

These values were derived from the suggested formula of $B = 1.4 + \rho$, where ρ is the utilization (Morton & Pentico, 1993).

5.2. Evaluation of the ATCSR heuristic using grid approach

The flowchart in Fig. 2 describes the grid approach algorithm. In this algorithm, we apply every combination of k_1 , k_2 and k_3 grid value to ATCSR index to generate multiple schedules, from which we choose the one with the minimum total weighted tardiness.

5.3. Experimental design for grid approach

In this section, we describe the experimental setup for comparing our ATCSR algorithm with the other algorithms using a grid approach. To evaluate the performance of our approach compared to other approaches, we solve $Pm|r_{j,s_{jk}}|\sum w_j T_j$ problem instances and compare the results with each of the following:

1. Earliest due date (EDD).
2. Weighted shortest processing time (WSPT).
3. Apparent tardiness cost with setups (ATCS) by Lee and Pinedo (1997).
4. Batch apparent tardiness cost with setups using a batch size of 1 (BATCS) by Mason et al. (2002).
5. Batch apparent tardiness cost with setups using a batch size of 1 modified (BATCSmod).
6. X-RMmod, modified from the X-RM index given by Morton and Pentico (1993).

We used a ‘Design of Experiments’ approach to generate a central composite design (Montgomery, 2001) described below. The following six factors (First four from Lee & Pinedo, 1997) are assumed to characterize a problem instance. Hence, they are expected to have an impact on the solution for a particular problem instance.

1. Job machine factor, $\mu = n/m$, which is the average number of jobs processed per machine.
2. Setup severity factor, $\eta = \bar{s}/\bar{p}$, which characterizes the effect of setup times on the schedule.
3. Due date tightness factor, $\tau = 1 - (\bar{d}/C_{\max})$, which characterizes how tight the due dates are. The schedule is tight if τ is close to 1 and loose if τ is close to 0.
4. Due date range factor, $R = (d_{\max} - d_{\min})/C_{\max}$, which is a measure of how spread out the due dates are as compared to the makespan.

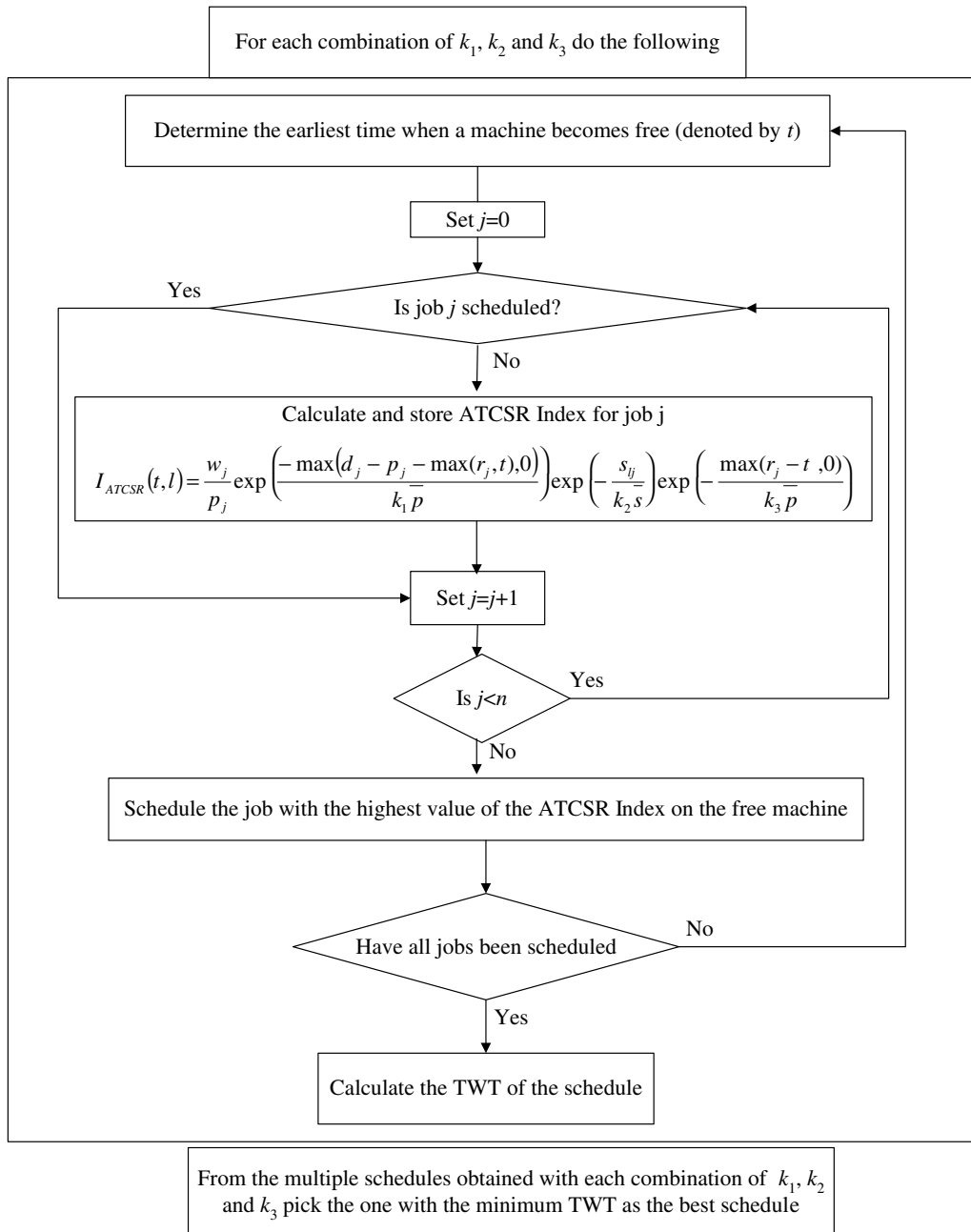
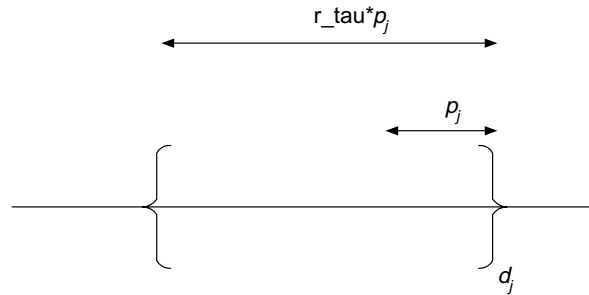


Fig. 2. Evaluation of ATCSR grid approach.

5. Job availability factor, $J_a = (\text{number of jobs available at time } 0)/n$, which is a measure of the fraction of jobs available at time 0.
6. Ready time tightness factor, r_{τ} , is a measure of how closely the ready times are placed to the due dates.

The ready time r_j is generated with uniform probability in the range $[d_j - r_{\tau} p_j, d_j]$, as indicated by the brace brackets in Fig. 3. If $d_j - r_{\tau} p_j$ is less than 0, a range of $[0, d_j]$ is used for ready time generation.

The responses for the experiment are the ratios of total weighted tardiness value given by the approaches to that given by the BATCS rule. BATCS was chosen as the reference because it was the best available rule that considered setup times and selected amongst ready and non-ready jobs.

Fig. 3. Explanation of r_tau factor.

5.4. Levels of factors

A central composite design, with the six factors described above, was employed. Since we have interest in any possible combination of the factors varying in a certain realistic range, we use a face-centered central composite design with $\alpha = 1$ (Montgomery, 2001). With 64 factorial points, 12 axial points and 4 center points for each replication, and 10 replications, there are 800 problem instances in total. Design Expert software (Stat-Ease, 2000) was used for data analysis. The ranges of the factors are shown in Table 2.

5.5. Problem instance data generation

The method for problem instance data generation which has been adapted from Lee and Pinedo (1997) is described below.

1. The number of machines is set to 5 ($m = 5$) and then the number of jobs is determined by using the value of the job machine factor μ , using $n = m * \mu$.
2. The processing times p_j are uniformly distributed over the interval $[50, 150]$.
3. The weights for the jobs w_j are uniformly distributed integers in the interval $[1, 10]$.
4. The mean setup time $\bar{s}$ is calculated by using the value of setup severity factor $\bar{s} = \eta * \bar{p}$. Further, setup times are uniformly distributed over the range $[0, 2\bar{s}]$.
5. The makespan is estimated by $C_{\max} = (\beta\bar{s} + \bar{p})\mu$, where β is the coefficient accounting for the increase in makespan due to setup times, which is given by $\beta = 0.4 + 10/\mu^2 - \eta/7$ (Lee & Pinedo, 1997).
6. The mean due date is calculated using $\bar{d} = C_{\max}(1 - \tau)$.
7. The due dates are uniformly distributed over the interval $[(1 - R)\bar{d}, \bar{d}]$ with probability τ and uniformly distributed over the interval $[\bar{d}, \bar{d} + (C_{\max} - \bar{d})R]$ with probability $(1 - \tau)$.
8. The Job availability factor J_a is used to randomly assign ready times of 0 if the value of a random number is less than J_a . The remaining jobs are assigned ready times uniformly distributed in the range $[d_j - r_tau * p_j, d_j]$ where r_tau is the ready time tightness factor. If $d_j - r_tau * p_j$ is less than 0, a range of $[0, d_j]$ is used. Thus, the ready time tightness factor defines the time by which the job could be ready before the due date.

Table 2
Values of factors for experimentation

Factors	Low level	Center point	High level
Job machine factor μ	11.000	19.000	27.000
Setup severity factor η	0.020	1.010	2.000
Due date tightness factor τ	0.300	0.600	0.900
Due date range factor R	0.250	0.630	1.000
Job availability factor J_a	0.200	0.500	0.800
Ready time factor r_tau	1.000	5.500	10.000

5.6. Performance of ATCSR grid approach

In this section we present the results of our experimentation for the performance of the ATCSR grid approach with respect to other algorithms. The grid approach was used for different rules and the ratio of the total weighted tardiness obtained by each of the rules over the total weighted tardiness obtained by the BATCS (base rule) was recorded. The average, best and worst case performance are shown in the grid approach portion of Tables 3–5 respectively.

The cases were grouped according to levels of factors such as the job machine factor, setup severity factor etc. For example, results for the job machine factor $\mu = 11$ indicate the result is the average of the ratios of total weighted tardiness value of an algorithm with the total weighted tardiness value of the BATCS rule by considering all the cases for which $\mu = 11$. Similarly, the best (worst) case performance table gives the minimum (maximum). In each table, the best performance amongst all the rules has been shown in bold.

We observe from Tables 3–5 that the ATCSR grid approach performs better than all the other approaches for the average, worst and best cases. In terms of average performance, the ATCSR grid approach, whose average ratio over BATCS grid approach is 0.80, is 14.89% better than the second best approach X-RMmod which has an average ratio as 0.94. For the best case, the performance of ATCSR and the X-RMmod grid approach are very close. However, the worst ratio of ATCSR grid approach is 1.39, which is much better than that of X-RMmod approach, which is 5.05. In Table 5, since the total weighted tardiness obtained by the BATCS grid approach is the denominator for all columns, the value in the BATCS grid column will always be 1, which does not mean it works better than ATCSR from the worst case point of view. Furthermore, amongst these 800 experiments, there are only 33 cases where the ATCSR grid approach performs worse than the BATCS grid approach; thus we conclude that the ATCSR grid approach works better than BATCS grid approach in general.

We observe from Table 3 that the difference between ATCS and the algorithms considering non-ready jobs such as the BATCS, X-RMmod and ATCSR is higher when the setup severity is high. This is due to the setup reduction achieved by waiting for non-ready jobs.

We also notice that the BATCSmod rule performs slightly better than the BATCS rule since 12 of 18 numbers in column 6 are less than 1 in Table 3, which means BATCSmod rule achieves lower total weighted tardiness for more problem instances than BATCS rule.

5.7. Distribution of scaling parameters

To check the frequency of occurrence of the scaling parameters and to ensure the range of the grid was adequate, we constructed a histogram of their values. They are shown in Figs. 4–6. From these results, we conclude that our grid is reasonable.

6. ATCSR algorithm using empirically based formula approach

The ATCSR grid approach described in the previous section calculates multiple schedules and hence consumes considerable computational time and resources. In order to reduce the computational effort, but hopefully still provide a good schedule, regression equations for each of the scaling parameters were developed using the scaling values selected in the grid approach. This variation of the algorithm using the ATCSR index is referred to as the ATCSR formula approach. The steps involved in generating the regression equations are given below:

1. Generate problem instances for the design points in the central composite design.
2. Measure the realized values of the problem instance characteristics.
3. For each of the problem instances solved, store the best values of k_1 , k_2 and k_3 .
4. Perform regression using realized values of the problem instance characteristics as regressors and the scaling parameters as the response.

Table 3
Average performance of approaches compared to BATCS grid approach

Factor	Grid approach							Formula approach			
	EDD	WSPT	ATCS	BATCS	BATCSmod	X-RMmod	ATCSR	ATCS formula	BATCS formula	X-RMmod formula	ATCSR formula
<i>Jobs/mc</i>											
$\mu = 11$	6.38	5.19	1.34	1.00	0.96	1.03	0.83	9.97	12.54	10.21	0.95
$\mu = 19$	5.71	4.46	0.96	1.00	1.03	0.81	0.70	2.05	4.34	2.02	0.78
$\mu = 27$	6.01	4.52	0.98	1.00	0.98	0.89	0.81	5.80	8.53	5.82	0.86
<i>Setup severity</i>											
$\eta = 0.02$	1.39	0.96	0.80	1.00	0.99	0.78	0.78	1.06	1.56	1.06	0.80
$\eta = 1.01$	5.67	4.43	0.98	1.00	1.03	0.83	0.71	2.07	4.36	2.04	0.80
$\eta = 2$	11.08	8.76	1.53	1.00	0.95	1.14	0.85	14.70	19.51	14.97	1.01
<i>Due date tightness</i>											
$\tau = 0.3$	9.46	7.50	1.32	1.00	0.94	0.94	0.68	14.97	20.68	15.29	0.83
$\tau = 0.6$	5.56	4.30	0.92	1.00	1.04	0.81	0.69	1.22	2.42	1.12	0.77
$\tau = 0.9$	3.06	2.28	1.02	1.00	1.00	0.99	0.96	1.15	1.21	1.12	0.99
<i>Due date range</i>											
$R = 0.25$	4.58	3.28	1.06	1.00	0.97	0.95	0.89	2.97	4.12	3.01	0.94
$R = 0.63$	10.56	8.84	1.35	1.00	1.01	0.92	0.58	18.64	26.13	18.87	0.76
$R = 1$	5.22	4.62	1.02	1.00	0.93	0.87	0.70	1.39	1.69	1.24	0.83
<i>% jobs available</i>											
$J_a = 0.2$	3.15	2.48	1.26	1.00	0.98	0.98	0.83	2.29	3.79	2.51	0.88
$J_a = 0.5$	5.57	4.37	0.97	1.00	1.04	0.83	0.71	2.04	4.35	2.01	0.79
$J_a = 0.8$	9.37	7.27	1.07	1.00	0.95	0.94	0.80	13.48	17.28	13.52	0.93
<i>Ready time tightness</i>											
$r_{\tau} = 1$	4.64	3.66	1.09	1.00	0.99	0.96	0.88	4.82	6.53	4.86	0.95
$r_{\tau} = 5.5$	5.56	4.34	0.95	1.00	1.02	0.82	0.71	2.03	4.28	2.02	0.79
$r_{\tau} = 10$	7.88	6.10	1.24	1.00	0.95	0.96	0.76	10.96	14.57	11.17	0.86
<i>Overall</i>	6.14	4.79	1.13	1.00	0.98	0.94	0.80	6.71	9.45	6.79	0.89

Values represent average value of ratio of specified approach to BATCS grid approach value.

Table 4
Best case performance of approaches compared to BATCS grid approach

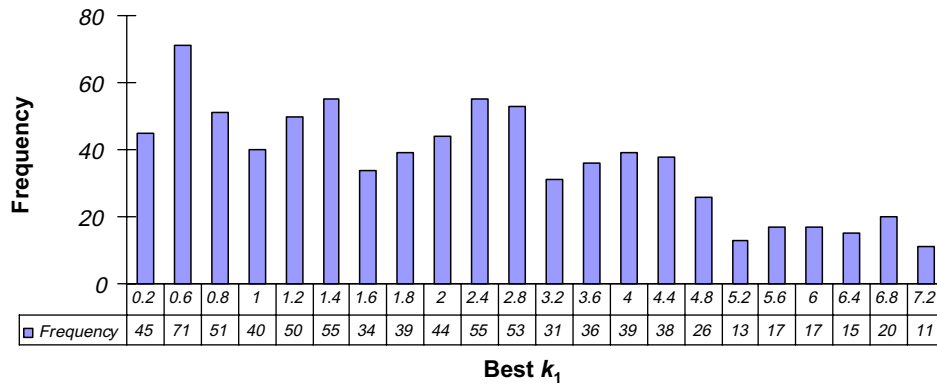
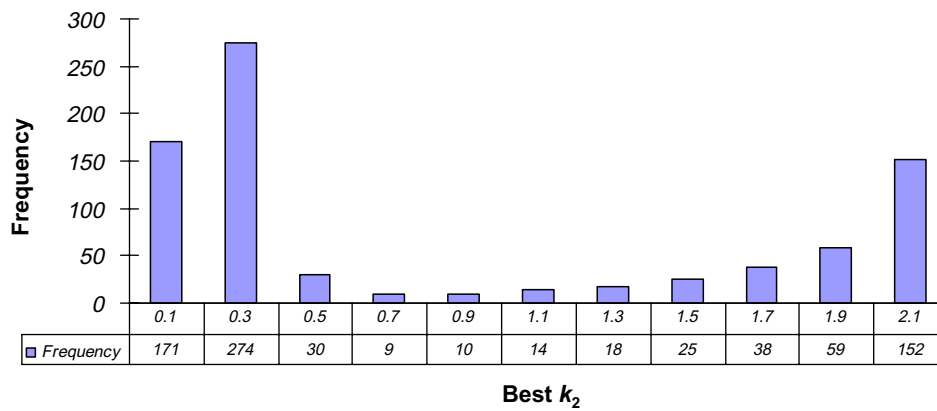
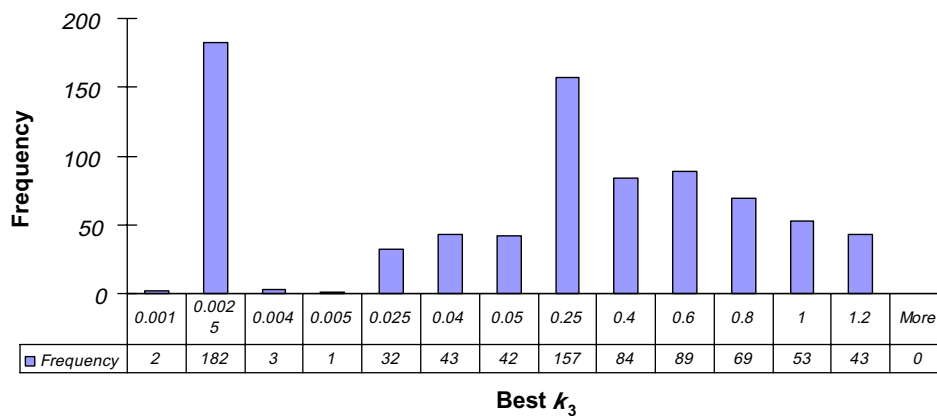
Factor	Grid approach							Formula approach			
	EDD	WSPT	ATCS	BATCS	BATCSmod	X-RMmod	ATCSR	ATCS formula	BATCS formula	X-RMmod formula	ATCSR formula
<i>Jobs/mc</i>											
$\mu = 11$	0.31	0.42	0.11	1.00	0.21	0.12	0.11	0.64	1.03	0.72	0.18
$\mu = 19$	0.76	0.64	0.34	1.00	0.59	0.33	0.14	0.57	1.09	0.54	0.25
$\mu = 27$	0.16	0.30	0.03	1.00	0.34	0.03	0.03	0.013	1.03	0.57	0.06
<i>Setup severity</i>											
$\eta = 0.02$	0.16	0.30	0.03	1.00	0.21	0.03	0.03	0.34	1.03	0.54	0.06
$\eta = 1.01$	2.56	1.87	0.37	1.00	0.59	0.33	0.14	0.63	1.11	0.68	0.25
$\eta = 2$	2.61	2.08	0.59	1.00	0.33	0.55	0.17	0.013	1.03	0.77	0.51
<i>Due date tightness</i>											
$\tau = 0.3$	0.16	0.30	0.03	1.00	0.30	0.03	0.03	0.6381	1.10	0.7159	0.06
$\tau = 0.6$	0.76	0.64	0.34	1.00	0.59	0.33	0.14	0.57	1.09	0.54	0.25
$\tau = 0.9$	0.31	0.76	0.17	1.00	0.21	0.15	0.14	0.013	1.03	0.72	0.18
<i>Due date range</i>											
$R = 0.25$	0.79	0.42	0.37	1.00	0.49	0.37	0.37	0.013	1.03	0.64	0.40
$R = 0.63$	0.16	0.30	0.03	1.00	0.30	0.03	0.03	0.34	1.20	0.54	0.06
$R = 1$	0.31	0.63	0.17	1.00	0.21	0.15	0.14	0.64	1.10	0.70	0.18
<i>% jobs available</i>											
$J_a = 0.2$	0.38	0.30	0.24	1.00	0.52	0.25	0.23	0.013	1.04	0.64	0.25
$J_a = 0.5$	0.76	0.64	0.34	1.00	0.66	0.33	0.14	0.57	1.09	0.54	0.25
$J_a = 0.8$	0.16	0.47	0.03	1.00	0.21	0.03	0.03	0.34	1.03	0.57	0.06
<i>Ready time tightness</i>											
$r_{\tau} = 1$	0.48	0.56	0.29	1.00	0.52	0.24	0.23	0.013	1.03	0.60	0.26
$r_{\tau} = 5.5$	0.76	0.64	0.34	1.00	0.59	0.34	0.14	0.57	1.09	0.54	0.25
$r_{\tau} = 10$	0.16	0.30	0.03	1.00	0.21	0.03	0.03	0.3377	1.03	0.5698	0.06
<i>Overall</i>	0.16	0.30	0.03	1.00	0.21	0.03	0.03	0.013	1.03	0.54	0.06

Values represent average value of ratio of specified approach to BATCS grid approach value.

Table 5
Worst case performance of approaches compared to BATCS grid approach

Factor	Grid approach							Formula approach			
	EDD	WSPT	ATCS	BATCS	BATCSmod	X-RMmod	ATCSR	ATCS formula	BATCS formula	X-RMmod formula	ATCSR formula
<i>Jobs/mc</i>											
$\mu = 11$	108.77	91.62	9.18	NA	2.69	5.05	1.39	279.07	293.43	276.62	2.59
$\mu = 19$	25.36	18.93	2.64	NA	1.80	1.28	1.01	43.26	102.49	46.49	1.12
$\mu = 27$	119.48	83.15	6.00	NA	2.11	2.62	1.01	260.57	409.00	261.68	1.64
<i>Setup severity</i>											
$\eta = 0.02$	3.23	3.02	1.10	NA	2.69	1.00	1.01	3.70	11.21	3.20	1.05
$\eta = 1.01$	25.36	18.93	2.64	NA	1.80	1.28	1.01	43.26	102.49	46.49	1.17
$\eta = 2$	119.48	91.62	9.18	NA	1.39	5.05	1.39	279.07	409.00	276.62	2.59
<i>Due date tightness</i>											
$\tau = 0.3$	119.48	91.62	9.18	NA	2.69	5.05	1.16	279.07	409.00	276.62	2.59
$\tau = 0.6$	15.02	18.59	2.34	NA	1.80	1.20	0.97	3.40	15.03	2.11	1.17
$\tau = 0.9$	24.84	19.69	3.77	NA	1.39	2.99	1.39	4.22	4.99	4.12	1.91
<i>Due date range</i>											
$R = 0.25$	40.34	38.97	7.63	NA	1.30	3.17	1.16	71.67	102.49	71.73	2.49
$R = 0.63$	119.48	91.62	9.18	NA	2.69	5.05	1.16	279.07	409.00	276.62	2.59
$R = 1$	24.84	19.69	3.77	NA	1.39	2.99	1.39	4.22	4.99	4.12	1.91
<i>% jobs available</i>											
$J_a = 0.2$	14.44	15.38	9.18	NA	2.69	3.24	1.39	19.99	47.80	24.20	1.91
$J_a = 0.5$	25.36	18.93	2.64	NA	1.80	1.28	1.01	43.26	102.49	46.49	1.17
$J_a = 0.8$	119.48	91.62	6.90	NA	2.11	5.05	1.16	279.07	409.00	276.62	2.59
<i>Ready time tightness</i>											
$r_{\tau} = 1$	38.11	35.96	3.37	NA	1.89	2.16	1.16	90.45	138.37	91.73	1.64
$r_{\tau} = 5.5$	25.36	18.93	2.64	NA	1.80	1.28	1.01	43.26	102.49	46.49	1.17
$r_{\tau} = 10$	119.48	91.62	9.18	NA	2.69	5.05	1.39	279.07	409.00	276.62	2.59
<i>Overall</i>	119.48	91.62	9.18	NA	2.69	5.05	1.39	279.07	409.00	276.62	2.59

Values represent average value of ratio of specified approach to BATCS grid approach value.

Fig. 4. Histogram of best values of k_1 .Fig. 5. Histogram of best values of k_2 .Fig. 6. Histogram of best values of k_3 .

6.1. Measurement of problem instance characteristics

In the regression, it is desirable to use the realized values of the factors instead of those used to generate the problem instance. This was done by measuring each factor as described below.

1. Job machine factor, $\mu = m/n$.
2. For calculation of setup severity factor, we calculate the sample mean of the processing times ($\bar{p}$) and setup times of the jobs ($\bar{s}$). Then, $\eta = \bar{s}/\bar{p}$.
3. For calculation of the due date tightness factor, we calculate the sample mean of all the due dates of jobs and estimate the makespan, $C_{\max}$. $C_{\max}$ is estimated by $\hat{C}_{\max} = (\beta\bar{s} + \bar{p})\mu$ where $\beta = 0.4 + 10/\mu^2 - \eta/7$ (Lee & Pinedo, 1997) is calculated by using the realized values of μ and η .
4. Due date tightness factor, $\tau = 1 - \bar{d}/\hat{C}_{\max}$.
5. Due date range is calculated by looking at the realized due dates of the jobs and finding the maximum ($d_{\max}$) and minimum ($d_{\min}$) due dates amongst them. Then $R = (d_{\max} - d_{\min})/\hat{C}_{\max}$.
6. Job availability factor, J_a = number of jobs ready at time 0/total number of jobs.
7. Ready time tightness factor was calculated using an iterative method.
 - i It is assumed that the ready times are distributed uniformly in the range $[d_j - r_{\tau}p_j, d_j]$ where r_{τ} is the specified ready time tightness factor. If $d_j - r_{\tau}p_j$ is less than 0, a range of $[0, d_j]$ is used. Jobs that have a value of $d_j - r_{\tau}p_j$ greater than 0 (see Fig. 7) are said to be in Set 1. Otherwise, a job is said to be in Set 2 (see Fig. 8). We derive the realized value of r_{τ} using this information.
 - ii Calculate the first approximation of r_{τ} by assuming all jobs to be in Set 1 (see Fig. 7). From Fig. 7, we see that $u = \frac{d_j - r_j}{r_{\tau}p_j}$, which is in percent of $r_{\tau}p_j$, the range of the ready time offset from the due date, is actually a uniformly distributed random variable in the range $[0, 1]$. Its expected value is 0.5. Thus, the expected value of u for all n jobs is $0.5n$. This leads to the following estimate of r_{τ} , $r_{\tau} = \frac{1}{0.5n} \sum \frac{d_j - r_j}{p_j}$.
 - iii For the second iteration, we calculate $d_j - r_{\tau}p_j$ for each job and separate the jobs into the two sets defined in (i) above.
 - iv For jobs in Set 1, u is defined as in (ii) above.
 - v For jobs in Set 2, we define u as $u = (d_j - r_j)/d_j$.
 - vi By summing all the u values and taking their expectation, we find that $r_{\tau} = \frac{\sum_{\text{Set 1}} ((d_j - r_j)/p_j)}{0.5n - \sum_{\text{Set 2}} ((d_j - r_j)/d_j)}$.
 - vii Using this new value of r_{τ} , we split the jobs into two sets (step iii) and calculate r_{τ} again. We continue doing this until the subsequent r_{τ} differ by at most 0.001. This is taken as the value for realized r_{τ} .

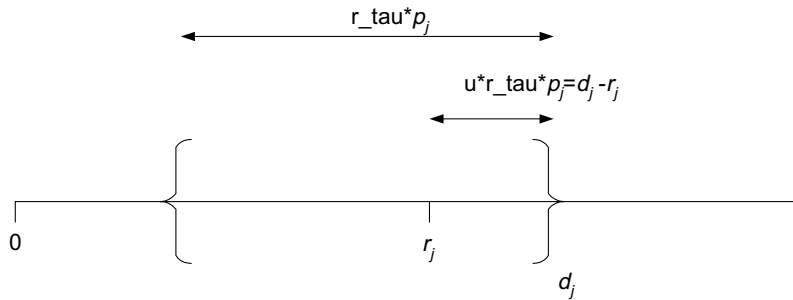


Fig. 7. Set 1 Jobs.

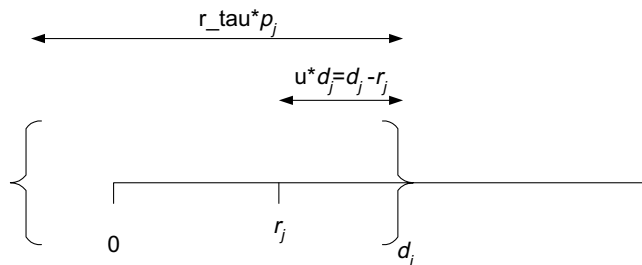


Fig. 8. Set 2 Jobs.

6.2. Results for regression equations

The regression analysis was performed and all assumptions were verified and met after a square-root transformation was applied. Resulting equations are shown below.

$$k_1 = (0.80 + 0.02\mu + 0.89\eta + 2.03\tau - 0.76R - 0.03r_{-}\tau - 0.45\eta^2 - 2.20r_{-}\tau^2 + 0.56\eta^*R + 0.02\eta^*r_{-}\tau)^2$$

R squared of the above model is 41.92% for k_1 . The normal probability plot for residuals is reasonable and the residuals versus fitted values graph is acceptable. The most significant factors for k_1 value are job machine factor, due date tightness and due date range.

$$k_2 = (1.35 - 1.13\eta + 0.34\eta^2)^2$$

R squared of the above model is 88.86% for k_2 . The normal probability plot for residuals is slightly curved but still acceptable. The residuals versus fitted values plot is acceptable. From the regression equation, the setup severity is shown to dominate the value of k_2 . This is not surprising because it adjusts the effect of the setup term in the ATCSR index.

$$k_3 = (0.28 + 0.74\eta + 1.08\tau - 0.02J_a - 0.02r_{-}\tau - 0.23\eta^2 - 1.06\tau^2 - 0.21\eta * \tau + 0.01\eta^*r_{-}\tau - 0.38\tau^*J_a)^2$$

R squared of the above model is 64.75% for k_3 . The normal probability plot is good except for slight distortion at the higher end. The residuals versus fitted values plot is acceptable. The most significant factors for the value of k_3 are setup severity, due date tightness, job availability and ready time tightness.

6.3. Evaluation of the ATCSR regression equations

In the formula approach, the same logic of Fig. 2 is used except that instead of determining k_1 , k_2 and k_3 using the grid, the realized values of the factors are determined and plugged into the regression equations above to determine k_1 , k_2 and k_3 .

To evaluate the performance of the regression equations, the ratio of the total weighted tardiness values obtained by the ATCSR grid approach to those obtained by ATCSR formula approach as the base rule were studied and are presented in ATCSR grid/ATCSR formula portion in Table 6. They are presented by problem instance characteristics to observe any possible variation in the performance of algorithms with problem instance characteristics.

From the last column of Table 6, we can see that the overall average ratio of the total weighted tardiness values obtained by ATCSR grid approach to those obtained by ATCSR formula approach is 0.91, which means that by applying the formula approach to save computational time, we incur an average increase in total weighted tardiness of about 9%. Note that the tighter the due date, the better the formula approach works. From Fig. 9, we can tell, for about 73% of the problem instances, ATCSR grid/ATCSR formula is larger than 0.9, which means that we generally do not give up very much in solution quality when using the empirical based formula approach. Note that it is possible for the ratio of ATCSR grid/ATCSR formula to be larger than one because the grid values of k_1 , k_2 and k_3 are discrete.

6.4. Performance of ATCSR formula approach

To determine how well the ATCSR formula approach performs, the ratio of the total weighted tardiness values obtained by applying ATCSR formula approach and other rules over those obtained by using BATCS grid approach (using parameter values given by Lee & Pinedo, 1997) as the base rule were studied. The average, best and worst case results for these ratios are shown in Tables 3–5 respectively.

We see from Table 3. that, the average ratio of ATCSR formula approach is 0.89, which is 11.25% worse than that of ATCSR grid approach (0.80), but better than all other grid approaches and much better than all of other formula approaches. The best ratio of ATCSR formula approach is 0.06, which is close to the best one of the ATCS formula approach, which is 0.013. As to the worst case, the ATCSR formula approach performs only inferior to the ATCSR grid approach, and better than all other approaches.

We note that the X-RMmod, ATCS, and BATCS formula average values are quite large for $\mu = 11$, $\eta = 2$, $\tau = 0.3$, $R = 0.63$, $J_a = 0.8$ and $r_{-}\tau = 10$. In fact, most values lie in the interval $[1, 2]$. However, there are some

Table 6
Average comparison for formula approach compared to grid approach

Factors	ATCS grid	BATCS grid	X-RMmod grid	ATCSR grid
	ATCS formula	BATCS formula	X-RMmod formula	ATCSR formula
<i>Jobs/mc</i>				
$\mu = 11$	0.65	0.56	0.60	0.89
$\mu = 19$	0.72	0.53	0.70	0.89
$\mu = 27$	1.20	0.70	0.75	0.93
<i>Setup severity</i>				
$\eta = 0.02$	0.76	0.75	0.75	0.96
$\eta = 1.01$	0.73	0.53	0.70	0.88
$\eta = 2$	1.08	0.51	0.60	0.87
<i>Due date tightness</i>				
$\tau = 0.3$	0.94	0.39	0.45	0.85
$\tau = 0.6$	0.75	0.54	0.73	0.88
$\tau = 0.9$	0.89	0.86	0.89	0.97
<i>Due date range</i>				
$R = 0.25$	1.06	0.72	0.77	0.95
$R = 0.63$	0.49	0.31	0.42	0.79
$R = 1$	0.71	0.67	0.69	0.88
<i>% jobs available</i>				
$J_a = 0.2$	1.19	0.63	0.71	0.94
$J_a = 0.5$	0.74	0.54	0.71	0.89
$J_a = 0.8$	0.65	0.62	0.64	0.87
<i>Ready time tightness</i>				
$r_\tau = 1$	1.17	0.65	0.71	0.93
$r_\tau = 5.5$	0.73	0.55	0.70	0.89
$r_\tau = 10$	0.67	0.60	0.63	0.89
<i>Overall average</i>	0.89	0.61	0.68	0.91
<i>Overall best case</i>	0.006	0.002	0.003	0.215
<i>Overall worst case</i>	77.72	0.97	1.11	1.13

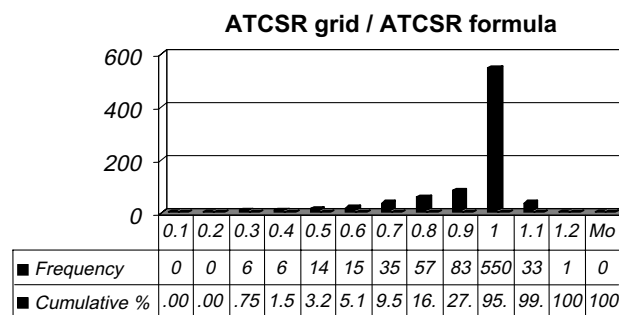


Fig. 9. Distribution of ratio of ATCSR grid to ATCSR formula approach.

instances where these heuristics perform quite poorly compared to the BATCS grid approach, as seen in Table 5.

6.5. Comparison between grid approach and formula approach for each rule

Table 6 presents the average performance for the comparison between the formula approaches and the respective grid approaches. For the algorithms other than ATCSR, the empirical based formulas proposed by the original author have been used. For the ATCS rule, the fitted functions proposed by Lee and Pinedo

(1997) are adopted. For the X-RMmod rule, parameter values suggested by Black, McKay, and Messimer (2004) have been used. These empirical formulas are not based on regression analysis. Table 6 gives us an idea of what we are losing by using formula compared to the grid approach. It illustrates that the relative loss is less by employing regression based formula.

From Table 6, we see the formula approach gives a total weighted tardiness higher by about 10–40% compared to the grid approach. The ATCSR formula approach works best amongst all the formula approaches, which only induces a 9% increase in total weighted tardiness value compared with respective ATCSR grid approach on the average.

The formula approach and grid approach are an excellent example of the tradeoff between computation time and quality of solution. The grid approach calculates the schedule for each combination of k_1 , k_2 and k_3 and thus generates multiple solutions. It then chooses the best solution amongst the ones generated. Thus, it explores the solution space by varying the parameter values in the formula.

The formula approach on the other hand uses information about the problem instance to determine values of k_1 , k_2 and k_3 . During the computation of the solution, it uses this equation to determine k_1 , k_2 and k_3 and calculates a single schedule using these values of parameters. Thus it is much faster than the grid approach. The significance of this was that one could use the formula approach to generate rough schedules for tool-groups as a first run in the Shifting Bottleneck Heuristic and then do the grid approach only for the top two or three toolgroups that are most likely to be the bottleneck.

7. Computation time versus solution quality

We compared the computation times for grid approach by running 10 replications of 80 problem instances. Table 7 provides the average computation time for each group of problem instances. The computation times

Table 7
Comparison for computation time for different grid algorithms (in seconds)

	ATCS	BATCS	BATCSmod	X-RMmod	ATCSR
<i>Jobs/mc</i>					
$\mu = 11$	0.19	0.22	0.23	2.62	3.19
$\mu = 19$	0.96	1.05	1.07	12.38	14.47
$\mu = 27$	2.58	2.69	2.78	34.13	37.10
<i>Setup severity</i>					
$\eta = 0.02$	1.38	1.44	1.49	18.21	19.93
$\eta = 1.01$	1.02	1.12	1.14	13.24	15.34
$\eta = 2$	1.37	1.44	1.49	18.18	20.00
<i>Due date tightness</i>					
$\tau = 0.3$	1.36	1.49	1.50	18.08	20.71
$\tau = 0.6$	1.02	1.12	1.14	13.23	15.40
$\tau = 0.9$	1.38	1.39	1.48	18.31	19.19
<i>Due date range</i>					
$R = 0.25$	1.41	1.46	1.52	18.64	20.13
$R = 0.63$	1.02	1.14	1.15	13.53	16.14
$R = 1$	1.39	1.49	1.52	18.32	20.44
<i>% jobs available</i>					
$J_a = 0.2$	1.33	1.44	1.49	18.11	20.33
$J_a = 0.5$	1.01	1.11	1.14	13.24	15.34
$J_a = 0.8$	1.41	1.44	1.49	18.28	19.60
<i>Ready time tightness</i>					
$r_{\tau} = 1$	1.37	1.44	1.49	18.19	19.99
$r_{\tau} = 5.5$	1.02	1.12	1.14	13.24	15.35
$r_{\tau} = 10$	1.38	1.44	1.49	18.20	19.93
<i>Overall</i>	1.31	1.38	1.43	17.33	19.15

Computation times for EDD and WSPT were approximately 0.002 s and computation times for formula approaches were approximately 0.006 s.

for EDD and WSPT were of the order of 0.002 s as there is no grid for these rules. X-RMmod and ATCSR have three scaling parameters and because the number of points in the grid is much larger these approaches take considerably more time. Moreover, ATCSR takes more time than X-RMmod as it has an exponential term and it calculates the index for all non-ready jobs.

Note that the computation time shows significant variation with the variation in the job machine factor, μ . This is expected as an increase in μ means more jobs need to be scheduled, which in turn means more calculations of index for every job to be scheduled. The computation times for the formula approaches were observed to be of the order of 0.006 s which is much lower than the grid approaches, as expected.

8. Conclusions and future research

ATCSR outperforms the compared algorithms in the literature for all cases considered. Thus, the opportunistic selection of non-ready jobs can reduce the total weighted tardiness of the schedule. This is the result of reduced setups or giving preference to high priority jobs arriving in the near future.

The grid approach can be used to get a very good solution using ATCSR. It generates multiple schedules by varying the parameters of the index and chooses the best amongst the schedules. On average, the ATCSR grid approach gives a total weighted tardiness lower by about 29% than the ATCS index grid approach, lower by about 20% than the BATCS index grid approach and about 15% lower than the modified X-RM index grid approach. Thus the ATCSR approach has the best average performance. X-RMmod is close in the best case, but has poor worst case performance.

The ATCSR formula approach can be used to get a rough-cut solution in a short time by using a single value of parameters. On average, the ATCSR formula approach gives results which are about 9% worse than the ATCSR grid approach.

This better estimate could further be used to reduce the size of the grid by calculating the schedules only for values in the vicinity of the regression estimate instead of the whole grid as we do now. This would substantially reduce the computation time for the grid approach. Considering a scheduling horizon only up to the smallest processing time of ready jobs or up to the next machine free time could also be studied. Calculating the scaling parameters for a horizon of time and then rolling ahead and generating new values for the scaling parameters could also be investigated.

References

- Abdul-Razaq, T. S., Potts, C. N., & Van Wassenhove, L. N. (1990). A survey of algorithms for the single machine total weighted tardiness scheduling problem. *Discrete Applied Mathematics*, 26, 235–253.
- Alidaee, B., & Rosa, D. (1997). Scheduling parallel machines to minimize total weighted and unweighted tardiness. *Computers and Operations Research*, 24, 775–788.
- Baker, K. R., & Bertrand, J. W. (1982). A dynamic priority rule for sequencing against due dates. *Journal of Operations Management*, 3, 37–42.
- Black, G. W., McKay, K. N., & Messimer, S. L. (2004). Predictive, stochastic and dynamic extensions to aversion dynamics scheduling. *Journal of Scheduling*, 7(4), 277–292.
- Carroll, D. C. (1965). Heuristic sequencing of jobs with single and multiple components. Ph.D. thesis. Cambridge, MA: Massachusetts Institute of Technology.
- Du, J., & Leung, J. Y. (1990). Minimizing total tardiness on one machine is NP-Hard. *Mathematics of Operations Research*, 15, 483–494.
- Eom, D. H., Shin, H. J., Kwun, I. H., Shim, J. K., & Kim, S. S. (2002). Scheduling jobs on parallel machines with sequence-dependent family set-up times. *The International Journal of Advanced Manufacturing Technology*, 19, 926–932.
- Fisher, M. L. (1981). The Lagrangian relaxation method for solving integer programming problems. *Management Science*, 27, 1–18.
- Huegler, P. A., & Vasko, F. J. (1997). A performance comparison of heuristics for the total weighted tardiness problem. *Computers and Industrial Engineering*, 32, 753–767.
- Lawler, E. L. (1979). Efficient implementation of dynamic programming algorithms for sequencing problems. Mathematisch Centrum, Amsterdam, Report BW 106.
- Lawler, E. L., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1982). Recent developments in deterministic sequencing and scheduling: A survey. In Lenstra Dempster & Kan Rinnooy (Eds.), *Deterministic and Stochastic Scheduling* (pp. 35–74). Dordrecht, Netherlands: D. Reidel Publishing Company.
- Lee, Y. H., & Pinedo, M. (1997). Scheduling jobs on parallel machines with sequence dependent setup times. *European Journal of Operational Research*, 100, 464–474.

- Luh, P. B., Hoiomt, D. J., Max, E., & Pattipati, K. R. (1990). Schedule generation and reconfiguration for parallel machines. *IEEE Transactions on Robotics and Automation*, 6(6), 687–696.
- Mason, S. J., Fowler, J. W., & Carlyle, W. M. (2002). A modified shifting bottleneck heuristic for minimizing total weighted tardiness in complex job shops. *Journal of Scheduling*, 5(3), 247–262.
- Montgomery, D. C. (2001). Design and analysis of experiments. New York: John Wiley and Sons.
- Morton, T., & Pentico, D. (1993). Heuristic scheduling systems: With applications to production systems and project management. New York: John Wiley and Sons.
- Morton, T. E., & Ramnath, P. (1995). *Guided forward search in tardiness scheduling of large one machine problems. Intelligent Scheduling Systems*. Massachusetts: Kluwer Academic Publishers.
- Park, Y., Kim, S., & Lee, Y. H. (2000). Scheduling jobs on parallel machines applying neural network and heuristic rules. *Computers and Industrial Engineering*, 38, 189–202.
- Pinedo, M. (2002). *Scheduling theory, algorithms, and systems*. Prentice Hall.
- Potts, C. N., & Van Wassenhove, L. N. (1985). A branch and bound algorithm for the total weighted tardiness problem. *Operations Research*, 33, 363–377.
- Rachamadugu, R. V., & Morton, T. E. (1982). Myopic heuristics for the single machine weighted tardiness problem. Working Paper, 30-82–83, GSIA. Pittsburgh, PA: Carnegie Mellon University.
- Rinnooy Kan, A. H. G., Lageweg, B. J., & Lenstra, J. K. (1975). Minimizing total costs in one-machine scheduling. *Operations Research*, 23, 908–927.
- Schrage, L., & Baker, K. R. (1978). Dynamic programming solution of sequencing problems with precedence constraints. *Operations Research*, 26, 444–449.
- Stat-Ease, Inc., Copyright 2000, Design-Expert Version 6.03, 2021 East Hennepin Ave., Suite 191, Minneapolis, MN 55413.
- Vepsalainen, A., & Morton, T. (1987). Priority rules for job shops with weighted tardiness costs. *Management Science*, 33, 1035–1047.